



What is apex triggers explain in detail with example

Sure! Let's go step by step so you get a **clear, detailed understanding of Apex Triggers** in Salesforce.

What is an Apex Trigger?

An **Apex Trigger** is a piece of code in Salesforce that **automatically executes before or after specific database events** occur on a Salesforce record (like insert, update, delete).

Think of a trigger as a "listener" that *waits for* changes in your data and then *acts* when those changes happen.

Triggers are written in **Apex**, Salesforce's proprietary Java-like programming language.

When do you use Triggers?

You use triggers when you want to:

- ✓ Automate tasks (e.g., auto-create related records).
- ✓ Validate complex business rules (beyond validation rules).
- ✓ Update related objects when something changes.
- ✓ Restrict or enforce custom logic during DML operations.

Trigger Events

Triggers can run **Before** or **After** specific DML events:

Event	Description
before insert	Before a new record is saved to the database.
before update	Before an existing record is updated in the database.
before delete	Before a record is deleted (useful for validations or restrictions).
after insert	After a new record has been saved to the database.
after update	After an existing record has been updated in the database.
after delete	After a record has been deleted.
after undelete	After a record has been restored from the Recycle Bin.

Key Points

- **Before Triggers:** Use them to update or validate record values before they are saved.
- **After Triggers:** Use them when you need the record **ID** (e.g., to create related records) or when making changes to other records.

Trigger Syntax

```
apex

trigger TriggerName on ObjectName (trigger_events) {
    // Your code logic
}
```

Example

```
apex

trigger AccountTrigger on Account (before insert, before update) {
    // Code here
}
```

Example Scenario

Problem:

Whenever a new Account is created, you want to **automatically create a related Contact** with the same name.

Trigger Example:

```
apex

trigger CreateContactOnAccountInsert on Account (after insert) {
    List<Contact> contactsToInsert = new List<Contact>();

    for (Account acc : Trigger.New) {
        Contact con = new Contact();
        con.FirstName = 'Primary';
        con.LastName = acc.Name;
        con.AccountId = acc.Id;
        contactsToInsert.add(con);
    }

    if (!contactsToInsert.isEmpty()) {
        insert contactsToInsert;
    }
}
```

Explanation:

- **after insert:** We use *after* so the **AccountId** is available.
- Loop over **Trigger.New** which contains the newly inserted Account records.
- For each Account, prepare a Contact.
- Insert all Contacts in bulk (important for efficiency).

Trigger Context Variables

Salesforce automatically provides special variables inside triggers:

Variable	Description
<code>Trigger.new</code>	List of new versions of the records (for insert/update).
<code>Trigger.old</code>	List of old versions of the records (for update/delete).
<code>Trigger.isInsert</code>	true if insert event.
<code>Trigger.isUpdate</code>	true if update event.
<code>Trigger.isDelete</code>	true if delete event.
<code>Trigger.isBefore</code>	true if before event.
<code>Trigger.isAfter</code>	true if after event.
<code>Trigger.newMap</code>	Map of IDs to new records.
<code>Trigger.oldMap</code>	Map of IDs to old records.

✓ Best Practices

- Always **bulkify** triggers (avoid writing logic assuming single record).
- Avoid **SOQL queries or DML statements inside loops**.
- Use **helper classes** to keep triggers clean and organized.
- **One Trigger per Object** is recommended (use logic dispatching to manage different scenarios).
- Write **Test Classes** (at least 75% code coverage required to deploy to production).

Summary

In simple terms:

Apex Triggers = *Automation code that runs automatically before or after data changes happen.*

They are a powerful way to enforce business logic and data integrity in Salesforce.

If you'd like, I can help you create more examples, best-practice patterns, or a helper class structure—just let me know!



Explain events in detail with examples

Absolutely—let's dive **deep** into **Trigger Events in Apex** so you understand clearly what each does, why they matter, and how to use them with examples.

Trigger Events Overview

In Apex, a **trigger event** determines *when* your trigger runs relative to the database operation.

There are **6 trigger events** you can use:

1. **before insert**
2. **before update**
3. **before delete**
4. **after insert**
5. **after update**
6. **after delete**
7. **after undelete**

Each of these events serves a **different purpose**.

Let's break them down one by one with examples:

before insert

When does it fire?

- Before a new record is saved to the database.

Use Case:

- You want to set default values or validate data *before* saving.

Example:

Scenario: Automatically set an Account's **Type** to **Prospect** if it's blank.

```
apex

trigger AccountBeforeInsert on Account (before insert) {
    for (Account acc : Trigger.New) {
        if (acc.Type == null) {
            acc.Type = 'Prospect';
        }
    }
}
```

This ensures the value is set before the record is saved.

before update

When does it fire?

- Before an existing record is updated in the database.

Use Case:

- Validate or modify record fields prior to committing the update.

Example:

Scenario: Prevent updating an Opportunity's **Amount** to more than \$1,000,000.

```
apex

trigger OpportunityBeforeUpdate on Opportunity (before update) {
    for (Opportunity opp : Trigger.New) {
        if (opp.Amount > 1000000) {
            opp.addError('Amount cannot exceed $1,000,000.');
        }
    }
}
```

`addError()` stops the DML operation and shows a message to the user.

3 before delete

✓ When does it fire?

- Before a record is deleted.

✓ Use Case:

- Prevent deletion based on custom rules.

✓ Example:

Scenario: Prevent deletion of any Account with **Type = Customer**.

```
apex

trigger AccountBeforeDelete on Account (before delete) {
    for (Account acc : Trigger.Old) {
        if (acc.Type == 'Customer') {
            acc.addError('You cannot delete Customer Accounts.');
        }
    }
}
```

Use `Trigger.Old` because the record is being removed, and no `Trigger.New` exists.

4 after insert

✓ When does it fire?

- After the record has been inserted into the database.

✓ Use Case:

- Create related records or make changes that need the record's ID.

✓ Example:

Scenario: Automatically create a Contact when a new Account is created.

```
apex

trigger AccountAfterInsert on Account (after insert) {
    List<Contact> newContacts = new List<Contact>();
    for (Account acc : Trigger.New) {
```

```

        Contact con = new Contact();
        con.FirstName = 'Primary';
        con.LastName = acc.Name;
        con.AccountId = acc.Id;
        newContacts.add(con);
    }
    insert newContacts;
}

```

Notice how `acc.Id` is available only *after insert*.

5 after update

✓ When does it fire?

- After an existing record has been updated in the database.

✓ Use Case:

- Update related records or log history.

✓ Example:

Scenario: When an Opportunity is marked as **Closed Won**, create a Case to onboard the customer.

```

apex

trigger OpportunityAfterUpdate on Opportunity (after update) {
    List<Case> casesToCreate = new List<Case>();
    for (Opportunity opp : Trigger.New) {
        Opportunity oldOpp = Trigger.OldMap.get(opp.Id);
        if (opp.StageName == 'Closed Won' && oldOpp.StageName != 'Closed Won') {
            Case newCase = new Case(
                Subject = 'New Customer Onboarding',
                AccountId = opp.AccountId,
                Status = 'New'
            );
            casesToCreate.add(newCase);
        }
    }
    insert casesToCreate;
}

```

`Trigger.OldMap` helps you compare the *previous value*.

6 after delete

✓ When does it fire?

- After the record has been deleted.

✓ Use Case:

- Delete or update related records.

✓ Example:

Scenario: Log deletion of Contacts into a custom object **Deleted_Record_Log__c**.

apex

```
trigger ContactAfterDelete on Contact (after delete) {
    List<Deleted_Record_Log__c> logs = new List<Deleted_Record_Log__c>();
    for (Contact con : Trigger.Old) {
        Deleted_Record_Log__c log = new Deleted_Record_Log__c(
            Record_Name__c = con.LastName,
            Record_Type__c = 'Contact',
            Deleted_On__c = System.now()
        );
        logs.add(log);
    }
    insert logs;
}
```

`Trigger.Old` has the deleted record details.

after undelete

✓ When does it fire?

- After a record is **restored from the Recycle Bin**.

✓ Use Case:

- Re-establish relationships or update related data when undeleting.

✓ Example:

Scenario: Log that an Account has been undeleted.

apex

```
trigger AccountAfterUndelete on Account (after undelete) {
    for (Account acc : Trigger.New) {
        System.debug('Account restored: ' + acc.Name);
    }
}
```

Summary of Events Table

Event	When it Runs	Can Modify Record Values?
before insert	Before saving a new record	✓ Yes
before update	Before updating an existing record	✓ Yes
before delete	Before deleting a record	✗ No (but can block)
after insert	After saving a new record	✗ No
after update	After updating an existing record	✗ No
after delete	After deleting a record	✗ No
after undelete	After restoring a deleted record from Recycle Bin	✗ No

✓ How to Choose the Right Event?

- **Need to validate or modify before saving?** → `before insert` / `before update`
- **Need record ID to create related records?** → `after insert`
- **Need to compare old and new values?** → `before update` / `after update`
- **Need to block deletes?** → `before delete`
- **Need to log deletions or clean up?** → `after delete`
- **Need to restore relationships?** → `after undelete`

If you'd like, I can create **more examples**, explain **Trigger Context Variables** in depth, or show **best practices for bulk processing**—just tell me!



Explain context variables with examples

Absolutely—let's **dive deep into Apex Trigger Context Variables** with clear explanations and examples. This is **one of the most common interview topics**, so understanding it well is important.

What Are Trigger Context Variables?

Context variables are special system-provided variables in Apex that give you details about:

- **Which operation is happening**
- **Which records are involved**
- **Whether your trigger is running in bulk**
- **Whether recursion is happening**

All context variables are available through the `Trigger` class.

List of Trigger Context Variables

Here are the most important ones you must know:

Variable	What it Means
<code>Trigger.isInsert</code>	Is this an insert operation? (true/false)
<code>Trigger.isUpdate</code>	Is this an update operation? (true/false)
<code>Trigger.isDelete</code>	Is this a delete operation? (true/false)
<code>Trigger.isBefore</code>	Is this a <i>before</i> trigger? (true/false)
<code>Trigger.isAfter</code>	Is this an <i>after</i> trigger? (true/false)
<code>Trigger.isUndelete</code>	Is this an undelete operation? (true/false)
<code>Trigger.New</code>	List of new versions of the records (insert/update/undelete)
<code>Trigger.NewMap</code>	Map of ID → new record (update/after insert/undelete)

Variable	What it Means
Trigger.Old	List of old versions of the records (update/delete)
Trigger.OldMap	Map of ID → old record (update/delete)
Trigger.size	Number of records that fired the trigger
Trigger.isExecuting	Is the trigger currently executing?

Let's understand each in detail **with examples**.

1 Trigger.isInsert

✓ What it does:

True if the trigger is fired by **insert operation**.

Example:

Log when records are inserted:

```
apex

trigger LeadTrigger on Lead (before insert, before update) {
    if (Trigger.isInsert) {
        System.debug('Inserting Leads...');
    }
}
```

2 Trigger.isUpdate

✓ What it does:

True if the trigger is fired by **update operation**.

Example:

Log when records are updated:

```
apex

trigger LeadTrigger on Lead (before insert, before update) {
    if (Trigger.isUpdate) {
        System.debug('Updating Leads...');
    }
}
```

3 Trigger.isDelete

✓ What it does:

True if records are being **deleted**.

Example:

apex

```
trigger AccountDeleteTrigger on Account (before delete) {
    if (Trigger.isDelete) {
        System.debug('Deleting Accounts...');
    }
}
```

4 Trigger.isBefore and Trigger.isAfter

✓ What it does:

Tells you whether the trigger runs **before** or **after** the database action.

Example:

apex

```
trigger ContactTrigger on Contact (before insert, after insert) {
    if (Trigger.isBefore) {
        System.debug('This is BEFORE insert.');
```

5 Trigger.isUndelete

✓ What it does:

True if the trigger runs when a record is **restored from Recycle Bin**.

Example:

apex

```
trigger AccountUndeleteTrigger on Account (after undelete) {
    if (Trigger.isUndelete) {
        System.debug('Undeleting Account...');
```

6 Trigger.New

✓ What it does:

List of new versions of records:

- Available in **insert**, **update**, and **undelete**.
- Not available in **delete**.

Example:

Set default field during insert:

apex

```
trigger AccountDefaultIndustry on Account (before insert) {
    for (Account acc : Trigger.New) {
        if (acc.Industry == null) {
            acc.Industry = 'Technology';
        }
    }
}
```

7 Trigger.Old

✓ What it does:

List of old versions of records:

- Available in **update** and **delete**.
- Not available in **insert**.

Example:

Prevent changing an Account's Name:

apex

```
trigger AccountNameLock on Account (before update) {
    for (Integer i = 0; i < Trigger.New.size(); i++) {
        if (Trigger.New[i].Name != Trigger.Old[i].Name) {
            Trigger.New[i].addError('Account name cannot be changed.');
```

Trigger.New[i] = New record

Trigger.Old[i] = Old record

8 Trigger.NewMap

✓ What it does:

A Map of record IDs to the **new version** of the records.

- Available in **update**, **after insert**, and **undelete**.

Example:

Compare old and new values:

apex

```
trigger OpportunityStageChange on Opportunity (before update) {
    for (Opportunity opp : Trigger.New) {
        Opportunity oldOpp = Trigger.OldMap.get(opp.Id);
        if (opp.StageName != oldOpp.StageName) {
            System.debug('Stage changed from ' + oldOpp.StageName + ' to ' + opp.StageName);
        }
    }
}
```

9 Trigger.OldMap

✓ What it does:

A Map of record IDs to **old version** of the records.

- Available in **update** and **delete**.

Example:

Log which records are being deleted:

```
apex

trigger ContactDeleteLog on Contact (before delete) {
    for (Contact con : Trigger.Old) {
        System.debug('Deleting Contact: ' + con.LastName);
    }
}
```

10 Trigger.size

✓ What it does:

Number of records that fired the trigger.

Example:

```
apex

trigger CaseBulkCount on Case (before insert) {
    System.debug('Number of Cases in Trigger: ' + Trigger.size);
}
```

11 Trigger.isExecuting

✓ What it does:

True if the trigger is currently executing.

- Useful in Apex classes shared by triggers and non-trigger logic.

Example:

```
apex

if (Trigger.isExecuting) {
    System.debug('Trigger is running.');
```

Example Combining Context Variables

Here is a **realistic example** combining several context variables:

✓ Scenario:

When an Opportunity is updated to **Closed Won**, and it wasn't **Closed Won** before, create a Case.

```
apex

trigger OpportunityClosedWonCase on Opportunity (after update) {
    if (Trigger.isAfter && Trigger.isUpdate) {
        List<Case> newCases = new List<Case>();
        for (Opportunity opp : Trigger.New) {
            Opportunity oldOpp = Trigger.OldMap.get(opp.Id);
            if (opp.StageName == 'Closed Won' && oldOpp.StageName != 'Closed Won') {
                newCases.add(new Case(
                    Subject = 'New Customer Onboarding',
                    AccountId = opp.AccountId,
                    Status = 'New'
                ));
            }
        }
        insert newCases;
    }
}
```

Quick Tips for Using Context Variables

- ✓ Use **Trigger.New** to modify records in **before triggers**
- ✓ Use **Trigger.Old** to compare previous values in **update triggers**
- ✓ Use **Trigger.NewMap** and **Trigger.OldMap** for fast lookup by ID
- ✓ Always design for **bulk processing** (**Trigger.size > 1**)
- ✓ Combine flags like **Trigger.isBefore && Trigger.isInsert** for clarity

If you'd like, I can prepare **more examples**, show **best practices for bulk triggers**, or help you with **trigger frameworks**—just let me know!



Explain recursive trigger in detail with examples

Absolutely—let's **deep-dive into Recursive Triggers in Salesforce Apex**, because this is a very important topic (especially in interviews and real-world development).

★ What is a Recursive Trigger?

✓ Definition:

A **recursive trigger** happens when your trigger logic causes the same trigger (or another trigger on the same object) to execute again **within the same transaction**—potentially **over and over**.

This can happen in two ways:

- 1 **Direct recursion** (your trigger updates the same object records it is running on)
- 2 **Indirect recursion** (your trigger updates related records, which then trigger updates back to the original records)

If you don't handle this carefully, you can get:

- **Maximum trigger depth exceeded**
- **Too many SOQL queries**
- **Too many DML statements**

Why Do Recursive Triggers Happen?

Here are common causes:

- You **update** records inside an **after trigger**.
- You **insert/update** related records whose triggers also update your object.
- You **update the same record** without any check to prevent repeated execution.

Example of Recursive Trigger

Scenario:

You have an **after update** trigger on Account that **updates the Account Name**:

```
apex

trigger AccountRecursive on Account (after update) {
    for (Account acc : Trigger.New) {
        acc.Name = acc.Name + ' Updated';
        update acc; // 🚩 This causes recursion
    }
}
```

✅ What happens here?

1. You update an Account record.
2. **after update** trigger fires.
3. It **updates the same record**, firing the trigger again.
4. Step 3 repeats until governor limits are hit.

Result:

```
sql

Maximum trigger depth exceeded.
```

Best Practices to Prevent Recursion

Salesforce does **NOT** have automatic recursion prevention.

You must implement your own logic.

✅ Approach 1: Use a Static Variable

- Static variables are initialized **only once per transaction**, and **retain their value throughout the transaction**.
- So you can use them as a “flag.”

Example of Preventing Recursive Trigger

Let's rewrite the above example properly.

Step 1: Create an Apex Helper Class with a Static Variable

```
apex

public class TriggerControl {
    public static Boolean isAccountUpdated = false;
}
```

Step 2: Update the Trigger

```
apex

trigger AccountRecursive on Account (after update) {
    if (TriggerControl.isAccountUpdated) {
        // Already updated in this transaction, do nothing
        return;
    }
    TriggerControl.isAccountUpdated = true;

    List<Account> accountsToUpdate = new List<Account>();
    for (Account acc : Trigger.New) {
        Account updatedAcc = new Account(
            Id = acc.Id,
            Name = acc.Name + ' Updated'
        );
        accountsToUpdate.add(updatedAcc);
    }
    update accountsToUpdate;
}
```

Explanation:

- On first execution, **isAccountUpdated** is **false**, so your logic runs and updates the Account.
- The flag is set to **true**.
- When the update fires the trigger again, it immediately **returns**—so no infinite recursion.

Indirect Recursion Example

Scenario:

- You have a trigger on **Opportunity** that updates the parent **Account**.
- You also have a trigger on **Account** that updates related **Opportunity** records.

This creates a loop:

1. Update Opportunity → triggers Account update

2. Account update → triggers Opportunity update

3. Back to Step 1.

✓ **Solution:**

Use static flags to detect whether your trigger has already performed its action.

Example of Indirect Recursion Prevention

Opportunity Trigger:

```
apex

public class OpportunityTriggerControl {
    public static Boolean isOpportunityTriggerRun = false;
}

trigger OpportunityUpdateTrigger on Opportunity (after insert, after update) {
    if (OpportunityTriggerControl.isOpportunityTriggerRun) {
        return;
    }
    OpportunityTriggerControl.isOpportunityTriggerRun = true;

    Set<Id> accountIds = new Set<Id>();
    for (Opportunity opp : Trigger.New) {
        if (opp.AccountId != null) {
            accountIds.add(opp.AccountId);
        }
    }
    List<Account> accountsToUpdate = [SELECT Id, Description FROM Account WHERE Id IN
:accountIds];
    for (Account acc : accountsToUpdate) {
        acc.Description = 'Updated from Opportunity trigger';
    }
    update accountsToUpdate;
}
```

Account Trigger:

```
apex

public class AccountTriggerControl {
    public static Boolean isAccountTriggerRun = false;
}

trigger AccountUpdateTrigger on Account (after update) {
    if (AccountTriggerControl.isAccountTriggerRun) {
        return;
    }
    AccountTriggerControl.isAccountTriggerRun = true;

    List<Opportunity> oppsToUpdate = new List<Opportunity>();
    for (Account acc : Trigger.New) {
        Opportunity opp = new Opportunity(
            Name = 'New Opportunity from Account Trigger',
            StageName = 'Prospecting',
            CloseDate = Date.today().addMonths(1),
            AccountId = acc.Id
        );
        oppsToUpdate.add(opp);
    }
    insert oppsToUpdate;
}
```

✓ Outcome:

- Each trigger only runs its logic **once per transaction**.

Additional Best Practices to Avoid Recursion

- ✓ Use **before triggers** to avoid DML if possible (e.g., modify `Trigger.New` directly).
- ✓ Always design for **bulk processing**—work on collections, not single records.
- ✓ Use **Trigger Handler classes** to encapsulate logic.
- ✓ Avoid DML inside loops.
- ✓ Carefully plan the sequence of related triggers.

Quick Recap

Recursive Trigger = Trigger logic that re-fires itself unintentionally.

✓ How to avoid it:

- Use **Static Variables** as flags.
- Use **before triggers** to change fields without DML.
- Design with **bulk-safe logic**.
- Avoid cross-object updates that create loops.

If you'd like, I can help you **set up a trigger framework** or provide **more advanced patterns for recursion control**—just let me know!



Trigger framework in detail

Great—let's do a **deep dive into a Trigger Framework in Salesforce Apex**.

This is an **advanced and professional approach** to organizing trigger logic cleanly, especially for:

- ✓ Large orgs with complex triggers
- ✓ Avoiding recursion, duplication, and governor limit issues
- ✓ Making triggers **maintainable and testable**

I'll walk you step by step:

★ What is a Trigger Framework?

A **Trigger Framework** is a *structured way of writing triggers* so you:

- Avoid putting logic directly inside the trigger.

- Keep triggers as *thin entry points*.
- Centralize logic in handler classes.
- Enable control over *order of operations*, recursion, and bulkification.

Think of it as the **MVC pattern applied to triggers**.

Why Use a Trigger Framework?

Problems with "classic" triggers:

- ✗ Logic spread across multiple triggers
- ✗ Hard to maintain or disable logic selectively
- ✗ High risk of hitting limits (SOQL/DML in loops)
- ✗ No easy way to control order or prevent recursion

✓ A Trigger Framework solves this by giving you:

- Separation of concerns
- Consistency across objects
- Bulk-safe processing
- Reusability
- Control over recursion

Components of a Typical Trigger Framework

Most frameworks have:

- 1 **Trigger** – Very thin, calls the handler class
- 2 **Trigger Handler Class** – Contains the business logic
- 3 **Helper Classes / Utility Classes** – Shared code (e.g., recursion control, common methods)
- 4 **Static Variables** – For recursion prevention

✨ Example: A Simple Trigger Framework

Let's walk step by step with a **real code example**.

1 Create the Trigger Handler Class

This class will hold your logic, separated by context (before insert, after update, etc).

```
apex

public class AccountTriggerHandler {
    public static Boolean isExecuting = false; // For recursion prevention

    public void beforeInsert(List<Account> newAccounts) {
        for (Account acc : newAccounts) {
```

```

        acc.Description = 'Created by Trigger Framework';
    }
}

public void afterInsert(List<Account> newAccounts) {
    // For example, create related records
    List<Contact> contactsToInsert = new List<Contact>();
    for (Account acc : newAccounts) {
        contactsToInsert.add(new Contact(
            LastName = 'Primary Contact',
            AccountId = acc.Id
        ));
    }
    if (!contactsToInsert.isEmpty()) {
        insert contactsToInsert;
    }
}

public void beforeUpdate(List<Account> newAccounts, Map<Id, Account> oldMap) {
    for (Account acc : newAccounts) {
        Account oldAcc = oldMap.get(acc.Id);
        if (acc.Name != oldAcc.Name) {
            acc.Description = 'Name was changed';
        }
    }
}
}

```

2 Write the Trigger

This is the **only trigger you will ever need on Account**.

See how clean it is:

```

apex

trigger AccountTrigger on Account (
    before insert,
    after insert,
    before update
) {
    if (AccountTriggerHandler.isExecuting) {
        return;
    }
    AccountTriggerHandler.isExecuting = true;

    AccountTriggerHandler handler = new AccountTriggerHandler();

    if (Trigger.isBefore) {
        if (Trigger.isInsert) {
            handler.beforeInsert(Trigger.new);
        } else if (Trigger.isUpdate) {
            handler.beforeUpdate(Trigger.new, Trigger.oldMap);
        }
    }
    if (Trigger.isAfter) {
        if (Trigger.isInsert) {
            handler.afterInsert(Trigger.new);
        }
    }
}

```

✓ Explanation:

- You call the correct method depending on the context.
- You pass **Trigger.new** and **Trigger.oldMap** as needed.

- You set `isExecuting = true` to prevent recursion.

Features You Can Add to This Framework

This is a base version. Production frameworks often include:

- ✓ Support for **all contexts**:
 - before insert
 - before update
 - before delete
 - after insert
 - after update
 - after delete
 - after undelete
- ✓ **Configurable logic** to enable/disable parts dynamically.
- ✓ **Centralized recursion control** (using a static utility class).
- ✓ **Order of execution** (e.g., `runBefore`, `runAfter` methods).
- ✓ **Error handling and custom exceptions.**

Example of a More Structured Framework

Here's a pattern called the **Trigger Factory Pattern**.

1 Generic Trigger Handler Base Class

You create an *abstract* base handler:

```
apex

public abstract class TriggerHandlerBase {
    public virtual void beforeInsert() {}
    public virtual void beforeUpdate() {}
    public virtual void beforeDelete() {}
    public virtual void afterInsert() {}
    public virtual void afterUpdate() {}
    public virtual void afterDelete() {}
    public virtual void afterUndelete() {}
}
```

2 Object-Specific Handler (e.g., Account)

apex

```
public class AccountTriggerHandler extends TriggerHandlerBase {
    public override void beforeInsert() {
        for (Account acc : (List<Account>) Trigger.new) {
            acc.Description = 'Created by Trigger Framework';
        }
    }

    public override void afterInsert() {
        List<Contact> contacts = new List<Contact>();
        for (Account acc : (List<Account>) Trigger.new) {
            contacts.add(new Contact(
                LastName = 'Created from Framework',
                AccountId = acc.Id
            ));
        }
        insert contacts;
    }
}
```

Generic Trigger

Your trigger becomes:

apex

```
trigger AccountTrigger on Account (
    before insert,
    after insert
) {
    TriggerHandlerBase handler = new AccountTriggerHandler();

    if (Trigger.isBefore) {
        if (Trigger.isInsert) handler.beforeInsert();
        if (Trigger.isUpdate) handler.beforeUpdate();
        if (Trigger.isDelete) handler.beforeDelete();
    }

    if (Trigger.isAfter) {
        if (Trigger.isInsert) handler.afterInsert();
        if (Trigger.isUpdate) handler.afterUpdate();
        if (Trigger.isDelete) handler.afterDelete();
        if (Trigger.isUndelete) handler.afterUndelete();
    }
}
```

Advantages:

- Clear separation of logic per context.
- Easy to read and maintain.
- Easy to add more objects by creating new handler classes.



Recursion Prevention in Frameworks

Pro Tip:

Use a centralized static utility class:

```
apex

public class TriggerUtility {
    private static Map<String, Boolean> executedTriggers = new Map<String, Boolean>();

    public static Boolean checkAndMarkTrigger(String triggerName) {
        if (executedTriggers.containsKey(triggerName)) {
            return true; // Already executed
        }
        executedTriggers.put(triggerName, true);
        return false; // First execution
    }
}
```

Then in your trigger:

```
apex

if (TriggerUtility.checkAndMarkTrigger('AccountTrigger')) {
    return; // prevent recursion
}
```

Summary of Benefits

- ✓ 1. Single Trigger per Object
- ✓ 2. Consistent and clear logic flow
- ✓ 3. Bulk-safe
- ✓ 4. Recursion control
- ✓ 5. Easy to maintain
- ✓ 6. Testable in isolation

Want Help Implementing?

If you like, I can help you:

- Create a **template** you can reuse across objects.
- Customize a framework for your org.
- Build **unit tests** to cover all scenarios.

Just let me know!



Trigger helper and handler class in detail with examples

Absolutely—let's go deep into **Trigger Helper** and **Trigger Handler** classes in Salesforce Apex. These are essential patterns for writing **clean, modular, maintainable trigger logic**.

I'll walk you step by step:

What Are Trigger Handler and Helper Classes?

✓ Trigger Handler Class

Directly manages **trigger context logic**—it decides what should happen in *before insert*, *after insert*, etc.

✓ Trigger Helper Class

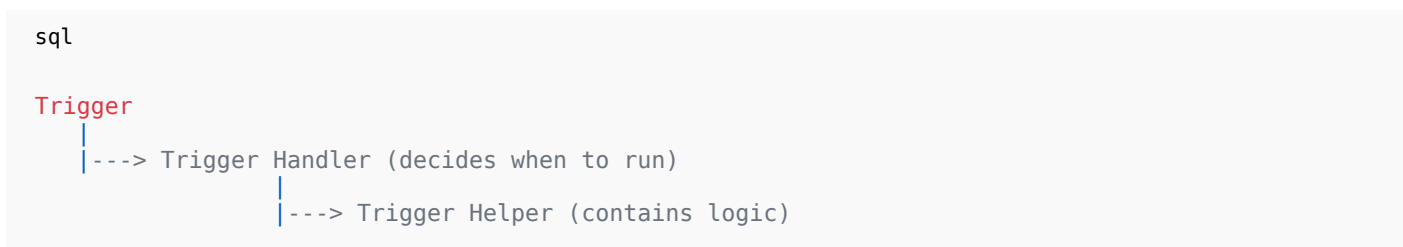
Contains **reusable business logic methods** called by the handler. Think of helpers as *utilities* or *services*.

Put simply:

- **Trigger:** Entry point, calls handler.
- **Handler:** Determines when to do what.
- **Helper:** Actually does the work.

Architecture

Here's the layered structure:



This clean separation helps you:

- ✓ Avoid logic inside triggers
- ✓ Reuse helpers in batch jobs or other Apex classes
- ✓ Keep your code organized

Example Scenario

Business Requirement:

- When an **Account** is created, set **Description** field.
- When **Account** name changes, create a **Note**.
- After insert, create a primary **Contact**.

□ Step by Step Example

1 Trigger

Keep the trigger itself *super thin*:

```
apex

trigger AccountTrigger on Account (
    before insert,
    before update,
    after insert
) {
    AccountTriggerHandler handler = new AccountTriggerHandler();

    if (Trigger.isBefore) {
        if (Trigger.isInsert) {
            handler.handleBeforeInsert(Trigger.new);
        }
        if (Trigger.isUpdate) {
            handler.handleBeforeUpdate(Trigger.new, Trigger.oldMap);
        }
    }

    if (Trigger.isAfter) {
        if (Trigger.isInsert) {
            handler.handleAfterInsert(Trigger.new);
        }
    }
}
```

✓ Explanation:

- No logic here.
- Simply calls **handler** methods.

2 Trigger Handler Class

This class **routes the trigger events**:

```
apex

public class AccountTriggerHandler {

    public void handleBeforeInsert(List<Account> newRecords) {
        AccountTriggerHelper.setDefaultDescription(newRecords);
    }

    public void handleBeforeUpdate(List<Account> newRecords, Map<Id, Account> oldMap) {
        AccountTriggerHelper.createNoteIfNameChanged(newRecords, oldMap);
    }

    public void handleAfterInsert(List<Account> newRecords) {
        AccountTriggerHelper.createPrimaryContact(newRecords);
    }
}
```

✓ Explanation:

- Each method handles a specific context.
- It simply delegates the work to the helper.

3 Trigger Helper Class

This class **contains the actual business logic**:

```
apex

public class AccountTriggerHelper {

    public static void setDescription(List<Account> accounts) {
        for (Account acc : accounts) {
            if (String.isBlank(acc.Description)) {
                acc.Description = 'Created via Trigger Framework';
            }
        }
    }

    public static void createNoteIfNameChanged(List<Account> accounts, Map<Id, Account> oldMap)
    {
        List<Note> notesToInsert = new List<Note>();

        for (Account acc : accounts) {
            Account oldAcc = oldMap.get(acc.Id);
            if (acc.Name != oldAcc.Name) {
                notesToInsert.add(new Note(
                    ParentId = acc.Id,
                    Title = 'Account Name Changed',
                    Body = 'Account name changed from "' + oldAcc.Name + '" to "' + acc.Name +
                        '"');
            }
        }

        if (!notesToInsert.isEmpty()) {
            insert notesToInsert;
        }
    }

    public static void createPrimaryContact(List<Account> accounts) {
        List<Contact> contactsToInsert = new List<Contact>();

        for (Account acc : accounts) {
            contactsToInsert.add(new Contact(
                LastName = 'Primary Contact',
                AccountId = acc.Id
            ));
        }

        if (!contactsToInsert.isEmpty()) {
            insert contactsToInsert;
        }
    }
}
```

✓ Explanation:

- All logic here is **bulkified**.
- You can call these helper methods elsewhere too (e.g., Batch Apex).

□ Benefits

✓ Trigger:

- Super clean.

- One trigger per object.

✓ **Handler:**

- Knows *when* to run logic.
- No business logic inside.

✓ **Helper:**

- Focused on *what* to do.
- Reusable from other Apex code.

✓ **Bulk-safe:**

- All methods take **List** inputs.

✓ **Testable:**

- You can unit-test helpers separately.

Recursion Handling Example

If you want to prevent **recursion**, use a static variable.
For example, in your handler:

```
apex

public class AccountTriggerHandler {
    private static Boolean isHandlerRunning = false;

    public void handleAfterInsert(List<Account> newRecords) {
        if (isHandlerRunning) return;

        isHandlerRunning = true;
        AccountTriggerHelper.createPrimaryContact(newRecords);
        isHandlerRunning = false;
    }
}
```

Quick Recap

Component	Role
Trigger	Entry point—only calls handler
Handler Class	Decides what logic runs in each context
Helper Class	Contains actual reusable business logic

Sample Use Cases

- Setting default field values

- Creating related records
- Sending emails
- Enforcing custom validations
- Updating other objects

Summary

- ✓ Clean
- ✓ Modular
- ✓ Reusable
- ✓ Easy to test
- ✓ Easy to maintain

If you'd like, I can help you:

- Create a **ready-to-use template** for any object.
- Show **unit test examples**.
- Help adapt this to your org.

Just share your use case!



Asynchronous apex in detail with examples

Great—let's **deep dive into Asynchronous Apex**, one of the most important concepts for writing efficient and scalable Salesforce applications.

I'll walk you through:

- ✓ What it is
- ✓ Why it's used
- ✓ All types with examples
- ✓ Best practices

What is Asynchronous Apex?

Asynchronous Apex allows you to run processes in the **background**, separate from the main transaction.

It helps you:

- Handle long-running operations
- Avoid hitting governor limits in synchronous context (e.g., DML limits, CPU time)
- Schedule tasks for later execution

- Process large data volumes in manageable chunks

Why Use Asynchronous Apex?

In Salesforce, every transaction has **governor limits**:

Context	Limit Example
Synchronous Apex	10 callouts per transaction
Asynchronous Apex	100 callouts per transaction

Asynchronous Apex raises these limits. Also, some use cases **require** async processing:

- Integrations with external systems
- Sending large batches of emails
- Performing complex calculations
- Updating thousands of records

Types of Asynchronous Apex

There are **5 main types**:

1. **Future Methods**
2. **Batch Apex**
3. **Queueable Apex**
4. **Scheduled Apex**
5. **Apex @future callouts**

We'll explain each with **examples**.

Future Methods

Use case: Perform a quick background operation that doesn't need to return data.

 *Key Features:*

- Annotated with **@future**
- Runs in its own thread
- Cannot return a value
- Can make HTTP callouts (**@future(callout=true)**)

 *Example:*

```
apex

public class FutureExample {
    @future
    public static void updateAccounts(List<Id> accountIds) {
        List<Account> accs = [SELECT Id, Description FROM Account WHERE Id IN :accountIds];
        for (Account acc : accs) {
            acc.Description = 'Updated asynchronously';
        }
        update accs;
    }
}
```

How to call:

```
apex

FutureExample.updateAccounts(new List<Id>{'001XXXXXXXXXXXX', '001YYYYYYYYYYYY'});
```

Batch Apex

Use case: Process large datasets (>50,000 records) in batches.

Key Features:

- Implements **Database.Batchable<SObject>**
- Breaks records into chunks (default batch size: 200)
- Supports querying and processing millions of records
- Each batch runs in its own transaction

Example:

```
apex

global class BatchAccountUpdate implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator('SELECT Id, Description FROM Account');
    }

    global void execute(Database.BatchableContext bc, List<Account> scope) {
        for (Account acc : scope) {
            acc.Description = 'Updated by Batch Apex';
        }
        update scope;
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Batch Job Completed.');
```

How to execute:

```
apex

BatchAccountUpdate batch = new BatchAccountUpdate();
```

```
Database.executeBatch(batch, 100);
```

3 Queueable Apex

Use case: More flexible alternative to future methods; supports complex processing and chaining jobs.

✓ Key Features:

- Implements **Queueable** interface
- Supports chaining another queueable job
- Supports non-primitive member variables
- Runs asynchronously

✓ Example:

```
apex

public class QueueableExample implements Queueable {
    public void execute(QueueableContext context) {
        List<Account> accs = [SELECT Id, Description FROM Account LIMIT 50];
        for (Account acc : accs) {
            acc.Description = 'Updated by Queueable Apex';
        }
        update accs;
    }
}
```

How to enqueue:

```
apex

System.enqueueJob(new QueueableExample());
```

✓ Chaining example:

```
apex

public class FirstJob implements Queueable {
    public void execute(QueueableContext context) {
        System.debug('First job running');
        System.enqueueJob(new SecondJob());
    }
}

public class SecondJob implements Queueable {
    public void execute(QueueableContext context) {
        System.debug('Second job running');
    }
}
```

4 Scheduled Apex

Use case: Run Apex code at a specific time or on a recurring schedule.

✓ Key Features:

- Implements **Schedulable** interface
- Can be scheduled via code or UI
- Cron expressions define recurrence

✓ Example:

```
apex

global class ScheduledExample implements Schedulable {

    global void execute(SchedulableContext context) {
        List<Account> accs = [SELECT Id, Description FROM Account LIMIT 10];
        for (Account acc : accs) {
            acc.Description = 'Updated by Scheduled Apex';
        }
        update accs;
    }
}
```

How to schedule:

```
apex

String cronExp = '0 0 12 * * ?'; // Every day at 12:00 PM
System.schedule('DailyAccountUpdate', cronExp, new ScheduledExample());
```

5 Future Callouts

Use case: Make HTTP callouts asynchronously.

✓ Example:

```
apex

public class FutureCalloutExample {

    @future(callout=true)
    public static void makeCallout() {
        HttpRequest req = new HttpRequest();
        req.setEndpoint('https://api.example.com/data');
        req.setMethod('GET');
        Http http = new Http();
        HttpResponse res = http.send(req);
        System.debug(res.getBody());
    }
}
```

Comparison of Types

Feature	Future	Queueable	Batch Apex	Scheduled Apex
Chaining Jobs	✗	✓	✓ (via finish)	✓ (via scheduling)
Supports Callouts	✓	✓	✓	✓

Feature	Future	Queueable	Batch Apex	Scheduled Apex
Handles Large Data	✗	Limited	✓	✓
Complex Logic	✗	✓	✓	✓
Synchronous Limits	✗	✗	✗	✗

🌟 Best Practices

- ✓ Always **bulkify** logic (work with lists, not single records)
- ✓ Avoid mixing DML and callouts in the same transaction
- ✓ Chain jobs when you need sequential processing
- ✓ Monitor jobs in **Setup** → **Apex Jobs**
- ✓ Use `System.enqueueJob()` for more flexibility than `@future`

🧠 When to Use What?

Use Case	Recommended Approach
Small background tasks	Future methods or Queueable
Sequential jobs	Queueable with chaining
Large data processing	Batch Apex
Scheduled daily or weekly jobs	Scheduled Apex
Callouts in background	Future(callout=true) or Queueable

💪 Example: A Complete Flow

Imagine this scenario:

- You want to **process 100k records nightly**
- Update them in batches
- Notify the admin when done

Solution:

1. **Scheduled Apex** triggers **Batch Apex**
2. **Batch Apex** processes records in chunks
3. **Finish method** sends an email

✓ That's the full picture of **Asynchronous Apex**!

If you want, I can help you:

- Write specific code samples
- Design a processing architecture

- Create test classes for async Apex

Just share your use case!



Explain future methods in detail with examples and use handler and helper class

Absolutely—let's **deep dive into Future Methods in Apex**, including **handler/helper class patterns with examples**.

What Are Future Methods?

A **Future Method** allows you to **run Apex code asynchronously**, in a separate thread, *after* the current transaction completes.

They are useful when:

- ✓ You want to **perform time-consuming operations** (e.g., callouts, large updates).
- ✓ You need to **avoid hitting synchronous governor limits**.
- ✓ You **don't need immediate results returned** to the user.

Key Characteristics of Future Methods

- Must be **static**.
- Must have **@future** annotation.
- Can only accept **primitive data types**, collections of primitives, or collections of IDs as parameters.
- Cannot return a value.
- Can't be called from another future method or batch job **execute()** method.
- Can be marked **@future(callout=true)** to do HTTP callouts.

When Should You Use Them?

- ✓ **DML operations in async context**
- ✓ **Callouts to external systems** after data is committed
- ✓ **Large-scale updates** where you need higher limits
- ✓ **Avoiding mixed DML errors** (e.g., updating setup and non-setup objects)

Syntax

```
apex

@future
public static void methodName(parameters) {
```

```
// your logic
}
```

Example – Simple Future Method:

```
apex

public class FutureExample {
    @future
    public static void updateContacts(List<Id> contactIds) {
        List<Contact> contacts = [SELECT Id, Description FROM Contact WHERE Id IN :contactIds];
        for (Contact c : contacts) {
            c.Description = 'Updated in future context';
        }
        update contacts;
    }
}
```

How to Call:

```
apex

FutureExample.updateContacts(new List<Id>{'003XXXXXXXXXXXXX', '003YYYYYYYYYYYYY'});
```



Governor Limits in Future Methods

Because Future Methods run asynchronously, you get **higher limits**:

Limit	Synchronous Context	Future Context
SOQL Queries	100	200
Records retrieved by SOQL	50,000	50,000
DML statements	150	150
HTTP Callouts	10	100

⚡ Future Methods for Callouts

When you need to call an external REST API, use:

```
apex

@future(callout=true)
public static void makeCallout() {
    Http h = new Http();
    HttpRequest req = new HttpRequest();
    req.setEndpoint('https://api.example.com/data');
    req.setMethod('GET');
    HttpResponse res = h.send(req);
    System.debug(res.getBody());
}
```

● Handler and Helper Pattern

In real-world projects, you often don't put logic directly inside your trigger or even directly in the Future Method class.

Instead, you use:

- **Handler Class:** The main class that orchestrates the logic.
- **Helper Class:** A utility class for reusable methods.

Example: Future Method with Handler and Helper Class

Scenario:

When an Account is updated, you asynchronously update related Contacts' descriptions.

1 Helper Class

This contains the *actual logic*.

```
apex

public class ContactHelper {
    public static void updateContactDescriptions(List<Id> contactIds) {
        List<Contact> contacts = [SELECT Id, Description FROM Contact WHERE Id IN :contactIds];
        for (Contact c : contacts) {
            c.Description = 'Updated via helper';
        }
        update contacts;
    }
}
```

2 Handler Class

This is the **future method**, which simply calls the helper.

```
apex

public class ContactHandler {
    @future
    public static void asyncUpdateContacts(List<Id> contactIds) {
        ContactHelper.updateContactDescriptions(contactIds);
    }
}
```

3 Trigger

This *fires when Accounts are updated* and calls the future method.

```
apex

trigger AccountTrigger on Account (after update) {
    Set<Id> contactIds = new Set<Id>();

    for (Account acc : Trigger.new) {
```

```

    if (acc.Some_Custom_Field__c == true) {
        // Collect related Contact Ids
        List<Contact> relatedContacts = [SELECT Id FROM Contact WHERE AccountId = :acc.Id];
        for (Contact c : relatedContacts) {
            contactIds.add(c.Id);
        }
    }
}

if (!contactIds.isEmpty()) {
    ContactHandler.asyncUpdateContacts(new List<Id>(contactIds));
}
}

```

✓ Flow of Control:

1. **Trigger fires** after update.
2. **Collect Contact IDs** related to updated Accounts.
3. **Call future method** in **ContactHandler**.
4. Future method **calls helper method**.
5. Helper method **updates Contacts asynchronously**.

Best Practices

- ✓ Always **bulkify your code** (e.g., work with lists, not single records).
- ✓ Avoid putting business logic directly in triggers or future method bodies—use **handlers and helpers**.
- ✓ Don't chain future methods (future calls can't call another future).
- ✓ Use **System.debug** or logs to trace async processes.
- ✓ Remember **limits on future calls per transaction** (50).

Quick Recap Table

Element	Purpose
Future Method	Run logic asynchronously
Handler Class	Entry point; defines @future
Helper Class	Contains reusable processing

- ✓ That's a clear, structured explanation of **Future Methods with handler and helper class examples**.

If you'd like, I can help you:

- Write test classes for this pattern.
- Show how to make callouts in this pattern.
- Compare future methods with Queueables.

Just let me know!



Explain batch async apex in detail with multiple examples and use handler and helper class also explains examples

Great—let's **deep dive into Batch Apex** in Salesforce:

🌟 Batch Apex Overview

Batch Apex allows you to **process large volumes of records asynchronously** by breaking them into manageable “chunks.”

It is the most common approach when you need to:

- ✓ Process **millions of records**
- ✓ Avoid hitting governor limits (SOQL, DML)
- ✓ Schedule long-running operations

◆ How Batch Apex Works

When you write a Batch Apex class, you implement the `Database.Batchable<SObject>` interface, which has **three methods**:

Method	Purpose
<code>start()</code>	Collects the records you want to process
<code>execute()</code>	Processes a “batch” of records (scope)
<code>finish()</code>	Runs logic after all batches complete (e.g., send email, chain batch)

● Basic Structure of Batch Apex

```
apex

global class MyBatchClass implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        // Return records to process
    }

    global void execute(Database.BatchableContext bc, List<SObject> scope) {
        // Process each batch
    }

    global void finish(Database.BatchableContext bc) {
        // Final steps
    }
}
```

✓ Important Points:

- `start()` returns either:
 - A **QueryLocator** (for large record sets—up to 50 million records).

- A List of SObjects.
- The records are automatically broken into “chunks” called **scope**.
- You can set **batch size** (1–2,000 records).
- Each batch runs in its own transaction—**limits are reset per batch**.

Example 1: Update All Contacts

Goal: Update all contacts to set a field asynchronously.

```
apex

global class UpdateContactsBatch implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator(
            'SELECT Id, Description FROM Contact'
        );
    }

    global void execute(Database.BatchableContext bc, List<Contact> scope) {
        for (Contact c : scope) {
            c.Description = 'Updated via Batch Apex';
        }
        update scope;
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Batch Processing Completed');
    }
}
```

✓ How to Execute:

```
apex

// Default batch size (200)
Database.executeBatch(new UpdateContactsBatch());

// Custom batch size (e.g., 100)
Database.executeBatch(new UpdateContactsBatch(), 100);
```

◆ Batch Apex Advantages

- ✓ Handles **millions of records**
- ✓ Limits are reset per batch (SOQL, DML, heap)
- ✓ Can schedule to run at night
- ✓ Allows monitoring via Apex Jobs in Setup

● Batch Apex with Handler and Helper Class

In real-life projects, you don’t want business logic in your Batch class itself.

Instead:

- **Batch class (Handler):** Orchestrates the batch lifecycle (**start**, **execute**, **finish**).
- **Helper class:** Contains reusable business logic.

Example 2: Batch Apex with Handler & Helper

Scenario: Update all Accounts with a custom field `Processed__c = true`.

1 Helper Class

Encapsulates processing logic:

```
apex

public class AccountHelper {
    public static void processAccounts(List<Account> accounts) {
        for (Account acc : accounts) {
            acc.Processed__c = true;
        }
        update accounts;
    }
}
```

2 Handler Class (Batch Class)

Implements the Batchable interface:

```
apex

global class AccountBatchHandler implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator(
            'SELECT Id, Processed__c FROM Account WHERE Processed__c = false'
        );
    }

    global void execute(Database.BatchableContext bc, List<Account> scope) {
        // Delegate processing to Helper
        AccountHelper.processAccounts(scope);
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Batch processing finished.');
```

Execute Batch:

```
apex

Database.executeBatch(new AccountBatchHandler(), 200);
```

Example 3: Sending Notification Emails in Finish Method

You can also perform additional tasks in `finish()`—like sending email notifications.

```
apex

global class AccountBatchWithNotification implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator(
            'SELECT Id FROM Account WHERE CreatedDate = LAST_N_DAYS:30'
        );
    }

    global void execute(Database.BatchableContext bc, List<Account> scope) {
        // Processing logic here
        for (Account acc : scope) {
            acc.Description = 'Batch processed recent Accounts';
        }
        update scope;
    }

    global void finish(Database.BatchableContext bc) {
        Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
        mail.setToAddresses(new String[] { 'admin@example.com' });
        mail.setSubject('Batch Processing Complete');
        mail.setPlainTextBody('The batch job completed successfully.');
```

```
Messaging.sendEmail(new Messaging.SingleEmailMessage[] { mail });
    }
}
```

Best Practices

- ✓ Always **bulkify** your code in `execute()`.
- ✓ Avoid large collections in memory—process the `scope` list only.
- ✓ Log progress in `finish()` for debugging.
- ✓ Use **helper classes** to encapsulate logic.
- ✓ Set **reasonable batch sizes** (200–500 is typical).

Governor Limits Recap (Per Execute Transaction)

Limit	Batch Apex Limit
SOQL Queries	100
Records retrieved by SOQL	50,000
DML Statements	150
Records processed	10,000
Heap Size	12 MB

- ✓ These limits **reset per batch**.

Quick Recap of Batch Apex Elements

Component	Role
<code>start()</code>	Collect records to process
<code>execute()</code>	Process each batch (scope)

Component	Role
finish()	Run final logic (e.g., notify, chain batches)

Example 4: Chaining Batches

You can **chain another batch in finish()**, e.g., process Accounts first, then Contacts:

```
apex

global class FirstBatch implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator('SELECT Id FROM Account');
    }

    global void execute(Database.BatchableContext bc, List<Account> scope) {
        for (Account acc : scope) {
            acc.Description = 'First batch processed';
        }
        update scope;
    }

    global void finish(Database.BatchableContext bc) {
        // Chain Second Batch
        Database.executeBatch(new SecondBatch());
    }
}

global class SecondBatch implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator('SELECT Id FROM Contact');
    }

    global void execute(Database.BatchableContext bc, List<Contact> scope) {
        for (Contact c : scope) {
            c.Description = 'Second batch processed';
        }
        update scope;
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Second batch completed.');
```

Summary

- ✓ **Batch Apex** = powerful tool for large data volumes.
- ✓ Use **handler and helper classes** for clean, reusable logic.
- ✓ Each batch runs with fresh governor limits.
- ✓ **finish()** is great for chaining or notifications.

Would you like help with:

- **Test classes** for batches?
- **Scheduling batch jobs**?
- **Batch + future method combinations**?

Just tell me!



Explain schedulable async apex in detail with multiple examples and use handler and helper class also explains examples

Absolutely—let's **deep dive into Schedulable Asynchronous Apex in Salesforce**, with **detailed examples, handler/helper classes**, and **practical use cases**.

What Is Schedulable Apex?

Schedulable Apex allows you to run Apex code **at a specific time or on a recurring schedule** (daily, weekly, monthly, etc.).

It's perfect for:

- ✓ Running nightly or hourly jobs
- ✓ Kicking off data processing (like Batch Apex) automatically
- ✓ Performing cleanup or maintenance tasks

Unlike Batch Apex, **Schedulable Apex itself doesn't automatically chunk records**.

You can, however, **call Batch Apex or Queueable Apex from inside Schedulable Apex**.

How Do You Use Schedulable Apex?

To schedule Apex, you:

1. **Implement** the `Schedulable` interface.
2. Provide the `execute()` method.
3. Schedule it via:
 - **`System.schedule()`** (programmatically)
 - Salesforce UI (Setup → Apex Classes → Schedule Apex)

Basic Syntax

```
apex

global class MyScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        // Code to run
    }
}
```

Example 1: Simple Scheduled Job

Goal: Update Accounts every night at 2 AM.

Helper Class

Encapsulate update logic:

```
apex

public class AccountUpdateHelper {
    public static void updateAccounts() {
        List<Account> accs = [
            SELECT Id, Description
            FROM Account
            WHERE Description = NULL
        ];
        for (Account acc : accs) {
            acc.Description = 'Updated by scheduler';
        }
        if (!accs.isEmpty()) {
            update accs;
        }
    }
}
```

Schedulable Handler Class

Implements **Schedulable** and calls helper:

```
apex

global class AccountUpdateScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        AccountUpdateHelper.updateAccounts();
    }
}
```

How to Schedule via Developer Console:

```
apex

String cronExp = '0 0 2 * * ?'; // Every day at 2 AM
String jobName = 'Nightly Account Update';
System.schedule(jobName, cronExp, new AccountUpdateScheduler());
```

✓ Cron Expression Format

SCSS

Seconds Minutes Hours Day_of_month Month Day_of_week Year(optional)

Examples:

- 0 0 0 * * ? → Midnight daily

- 0 0 2 * * ? → 2 AM daily
- 0 30 5 * * ? → 5:30 AM daily

Example 2: Scheduling a Batch Job

It is very common to **use Schedulable Apex to kick off Batch Apex**, because Batch Apex itself does not have a built-in scheduler.

Helper Class (Business Logic)

```
apex

public class ContactBatchHelper {
    public static void processContacts(List<Contact> contacts) {
        for (Contact c : contacts) {
            c.Description = 'Processed by Batch';
        }
        update contacts;
    }
}
```

Batch Apex Class

```
apex

global class ContactBatch implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator(
            'SELECT Id, Description FROM Contact WHERE LastModifiedDate = LAST_N_DAYS:7'
        );
    }

    global void execute(Database.BatchableContext bc, List<Contact> scope) {
        ContactBatchHelper.processContacts(scope);
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Batch completed.');
```

Schedulable Handler Class

Schedules the batch:

```
apex

global class ContactBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        ContactBatch batch = new ContactBatch();
        Database.executeBatch(batch, 200); // Batch size 200
    }
}
```

✓ How to Schedule It Programmatically

apex

```
String cronExp = '0 0 1 * * ?'; // 1 AM daily
System.schedule('Daily Contact Batch', cronExp, new ContactBatchScheduler());
```

✓ How to Schedule via UI

- Go to **Setup > Apex Classes**
- Click **Schedule Apex**
- Select **ContactBatchScheduler**
- Provide frequency and start time

Example 3: Chaining Batch Jobs via Scheduler

Suppose you have **two batch jobs that must run nightly**, in sequence.

Batch #1: Update Accounts

apex

```
global class BatchAccounts implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator('SELECT Id, Name FROM Account');
    }

    global void execute(Database.BatchableContext bc, List<Account> scope) {
        for (Account acc : scope) {
            acc.Description = 'Batch 1 completed';
        }
        update scope;
    }

    global void finish(Database.BatchableContext bc) {
        // Start second batch
        Database.executeBatch(new BatchContacts());
    }
}
```

Batch #2: Update Contacts

apex

```
global class BatchContacts implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator('SELECT Id, Name FROM Contact');
    }

    global void execute(Database.BatchableContext bc, List<Contact> scope) {
        for (Contact c : scope) {
            c.Description = 'Batch 2 completed';
        }
    }
}
```

```

    }
    update scope;
  }

  global void finish(Database.BatchableContext bc) {
    System.debug('Both batches finished.');
```

3 Scheduler to Start Batch #1

```

apex

global class BatchChainScheduler implements Schedulable {
  global void execute(SchedulableContext sc) {
    Database.executeBatch(new BatchAccounts());
  }
}
```

✓ Schedule It

```

apex

String cronExp = '0 0 23 * * ?'; // 11 PM daily
System.schedule('Nightly Chained Batches', cronExp, new BatchChainScheduler());
```

● Best Practices

- ✓ Always **move business logic into helper classes**
- ✓ Keep Schedulable Apex **small and focused**—just kick off processes
- ✓ Use **System.schedule()** for dynamic scheduling
- ✓ Document **cron expressions carefully**
- ✓ Monitor scheduled jobs in **Setup > Scheduled Jobs**

🔑 When to Use Schedulable Apex?

- ✓ Schedule periodic cleanup or processing
- ✓ Automate reporting or notifications
- ✓ Kick off batch jobs on a schedule
- ✓ Chain multiple processing steps overnight

🚀 Recap Table

Component	Purpose
Schedulable Interface	Makes a class schedulable
execute()	Runs when job is triggered
System.schedule()	Schedules the job programmatically

Component	Purpose
Batch/Queueable Apex	Can be called from schedulable Apex
Helper Classes	Encapsulate reusable business logic

Extra: Sample Cron Expressions

Schedule	Cron Expression
Every day at midnight	0 0 0 * * ?
Every day at 6 AM	0 0 6 * * ?
Every Monday at 9 PM	0 0 21 ? * MON
1st of every month at 1 AM	0 0 1 1 * ?
Every 15 minutes	0 0/15 * * * ?

Would you like help with:

- Test classes for Schedulable Apex?
- Combining Schedulable + Queueable?
- Chaining multiple schedulers?

Just tell me—I'll guide you step by step!



Explain queueable async apex in detail with multiple examples and use handler and helper class also explains examples

Absolutely—let's **deep dive into Queueable Asynchronous Apex**, including **detailed examples, handler/helper classes**, and best practices.

What is Queueable Apex?

Queueable Apex is an asynchronous job that:

- Runs in the background.
- Allows you to process **large volumes of data in smaller chunks**.
- Lets you **chain jobs** (one queueable can enqueue another).
- Provides more control and flexibility than **@future**.

Main Advantages over @future:

- Supports complex data types as parameters (collections, sObjects).
- Supports job chaining.

- Can see job status in the Apex Jobs page.

✓ Limitations:

- Each transaction can enqueue up to **50** jobs.
- Chained jobs can only be **one at a time** (i.e., you can't start multiple jobs in a single `execute()`).

Syntax Overview

To implement Queueable Apex:

```
apex

public class MyQueueableJob implements Queueable {
    public void execute(QueueableContext context) {
        // Your logic here
    }
}
```

You enqueue it using:

```
apex

ID jobId = System.enqueueJob(new MyQueueableJob());
```

✨ Example 1: Simple Queueable Job

Let's say you want to **update a batch of Accounts** asynchronously.

Helper Class

Encapsulates logic to update Accounts:

```
apex

public class AccountHelper {
    public static void updateDescriptions(List<Account> accs) {
        for (Account acc : accs) {
            acc.Description = 'Updated by Queueable Apex';
        }
        update accs;
    }
}
```

Queueable Handler Class

Implements `Queueable` and calls the helper:

apex

```
public class UpdateAccountsQueueable implements Queueable {
    private List<Account> recordsToUpdate;

    public UpdateAccountsQueueable(List<Account> accs) {
        this.recordsToUpdate = accs;
    }

    public void execute(QueueableContext context) {
        AccountHelper.updateDescriptions(recordsToUpdate);
    }
}
```

✓ How to Execute It:

apex

```
List<Account> accs = [SELECT Id FROM Account LIMIT 5];
System.enqueueJob(new UpdateAccountsQueueable(accs));
```

This will **asynchronously update 5 Accounts**.

✨ Example 2: Chaining Queueable Jobs

Let's say you want to:

1. Update Accounts.
2. When finished, update Contacts.

Helper for Contacts

apex

```
public class ContactHelper {
    public static void updateContacts(List<Contact> cons) {
        for (Contact c : cons) {
            c.Description = 'Updated by Queueable Apex';
        }
        update cons;
    }
}
```

Queueable Job to Update Contacts

apex

```
public class UpdateContactsQueueable implements Queueable {
    public void execute(QueueableContext context) {
        List<Contact> cons = [SELECT Id FROM Contact LIMIT 5];
        ContactHelper.updateContacts(cons);
    }
}
```

Queueable Job to Update Accounts and Chain

```
apex

public class UpdateAccountsAndChainQueueable implements Queueable {
    public void execute(QueueableContext context) {
        List<Account> accs = [SELECT Id FROM Account LIMIT 5];
        AccountHelper.updateDescriptions(accs);

        // Chain the next job
        System.enqueueJob(new UpdateContactsQueueable());
    }
}
```

✅ Enqueue the first job:

```
apex

System.enqueueJob(new UpdateAccountsAndChainQueueable());
```

This will:

1. Update Accounts.
2. When complete, **automatically** enqueue the job to update Contacts.

✨ Example 3: Processing Large Data Sets in Batches

Suppose you want to process **10,000 Leads** but not all in one go.
You can pass in a List of IDs and process smaller chunks.

Helper Class

```
apex

public class LeadHelper {
    public static void processLeads(List<Lead> leads) {
        for (Lead l : leads) {
            l.Description = 'Processed in Queueable Batch';
        }
        update leads;
    }
}
```

Queueable Class with Batching

```
apex

public class ProcessLeadsQueueable implements Queueable {
    private List<Id> leadIds;
    private Integer batchSize;
```

```
public ProcessLeadsQueueable(List<Id> ids, Integer batchSize) {
    this.leadIds = ids;
    this.batchSize = batchSize;
}

public void execute(QueueableContext context) {
    List<Lead> batch = [SELECT Id, Description FROM Lead WHERE Id IN :leadIds LIMIT
:batchSize];

    if (!batch.isEmpty()) {
        LeadHelper.processLeads(batch);

        // Remove processed IDs
        List<Id> remainingIds = new List<Id>();
        for (Id id : leadIds) {
            if (!batch.contains(id)) {
                remainingIds.add(id);
            }
        }

        // Chain next batch if any left
        if (!remainingIds.isEmpty()) {
            System.enqueueJob(new ProcessLeadsQueueable(remainingIds, batchSize));
        }
    }
}
```

✓ How to Execute:

apex

```
List<Id> allLeadIds = [SELECT Id FROM Lead LIMIT 1000].stream().map(l -> l.Id).collect();
System.enqueueJob(new ProcessLeadsQueueable(allLeadIds, 200));
```

This will:

- Process 200 Leads at a time.
- Chain the next job automatically until all are processed.

✨ Example 4: Passing Complex Parameters

Queueable can accept Lists, Maps, SObjects:

apex

```
public class ProcessOpportunitiesQueueable implements Queueable {
    private Map<Id, Decimal> oppAdjustments;

    public ProcessOpportunitiesQueueable(Map<Id, Decimal> adjustments) {
        this.oppAdjustments = adjustments;
    }

    public void execute(QueueableContext context) {
        List<Opportunity> opps = [SELECT Id, Amount FROM Opportunity WHERE Id IN
:oppAdjustments.keySet()];
        for (Opportunity opp : opps) {
            opp.Amount += oppAdjustments.get(opp.Id);
        }
        update opps;
    }
}
```

```
}  
}
```

✓ To Execute:

```
apex  
  
Map<Id, Decimal> adjustments = new Map<Id, Decimal>{  
    '006xx000001234A' => 5000,  
    '006xx000001234B' => -2000  
};  
System.enqueueJob(new ProcessOpportunitiesQueueable(adjustments));
```

✓ Best Practices

- ✓ Always move **business logic into helper classes**
- ✓ Keep Queueable Apex **transactional and focused**
- ✓ Avoid DML limits by processing reasonable batch sizes
- ✓ Use chaining for multi-step processes
- ✓ Use **Database.SaveResult** for error handling
- ✓ Monitor jobs in **Setup > Apex Jobs**

Queueable vs Future vs Batch

Feature	@future	Queueable Apex	Batch Apex
Complex Parameters	No	Yes	Yes
Chaining	No	Yes (one chain)	No (but can call Queueable in finish)
Chunk Large Data	No	Manual batching	Automatic batching
Monitoring Jobs	Limited	Detailed	Detailed
Schedule / Cron	No	No	Yes (with Scheduler)
Max Concurrent Jobs	50	50	5 concurrent batches

Quick Recap

Queueable Apex is your go-to solution for:

- Chaining jobs.
- Processing records in smaller units.
- Passing complex parameters.
- Better visibility in job monitoring.

Would you like help with:

- ✓ Writing test classes?
- ✓ Designing a multi-step queueable chain?
- ✓ Combining Queueable + Schedulable or Batch?

Just let me know—I'm here to help you build production-ready code step by step!



Invocable method

Absolutely—let's **deep dive into Invocable Methods in Salesforce Apex**, including:

- ✓ What they are
- ✓ Why you use them
- ✓ Syntax
- ✓ Examples

● What is an Invocable Method?

An **Invocable Method** is an Apex method you expose to:

- **Flows** (Flow Builder)
- **Process Builder** (deprecated but still in many orgs)
- Other declarative tools

This allows **admins and declarative users** to call Apex logic **without writing code themselves**.

✓ Use Cases:

- When you need custom logic in a Flow (e.g., complex record updates).
- When Process Builder can't do something out of the box.
- When you want to share logic with external systems via Flow Orchestration.

Key Rules

- Must be **public** or **global**.
- Must be **static**.
- Must be annotated with **@InvocableMethod**.
- Can accept a **single parameter**, which must be:
 - A **List of a custom Apex class** (recommended)
 - A **List of primitive types**
 - A **List of sObjects**
- Returns **void** or a List.

✨ Syntax Example 1: Simple Invocable Method

Let's say you want to update the **Description** field of Accounts from a Flow.

```
apex

public class AccountUpdater {
    @InvocableMethod(label='Update Account Descriptions' description='Updates Description field on Accounts')
    public static void updateDescriptions(List<Id> accountIds) {
        List<Account> accs = [SELECT Id FROM Account WHERE Id IN :accountIds];
        for (Account acc : accs) {
            acc.Description = 'Updated by Flow';
        }
        update accs;
    }
}
```

✓ How to use this:

- In Flow Builder:
 - Add an **Action**.
 - Search for **Update Account Descriptions**.
 - Pass in the List of Account Ids.
 - When the Flow runs, the Apex method updates them.

✨ Syntax Example 2: Using Custom Input Types

This is the **recommended approach** for clarity and flexibility.

Step 1: Define an Input Class

```
apex

public class AccountDescriptionInput {
    @InvocableVariable(required=true label='Account Id')
    public Id accountId;

    @InvocableVariable(required=true label='New Description')
    public String newDescription;
}
```

Step 2: Define the Invocable Method

```
apex

public class AccountDescriptionUpdater {
    @InvocableMethod(label='Update Descriptions with Input')
    public static void updateDescriptions(List<AccountDescriptionInput> inputList) {
        Map<Id, String> idToDesc = new Map<Id, String>();
        for (AccountDescriptionInput input : inputList) {
```

```

        idToDesc.put(input.accountId, input.newDescription);
    }
    List<Account> accs = [SELECT Id, Description FROM Account WHERE Id IN
:idToDesc.keySet()];
    for (Account acc : accs) {
        acc.Description = idToDesc.get(acc.Id);
    }
    update accs;
}
}

```

✓ How to use this:

- In Flow Builder:
 - Create a **Collection Variable** containing records with **accountId** and **newDescription**.
 - Pass the collection to the Action.
 - This method will update each Account accordingly.

✨ Example 3: Returning Data

Invocable Methods can also **return values**, for example, to pass back results to the Flow.

Input Class

```

apex

public class ContactInput {
    @InvocableVariable(required=true)
    public Id contactId;
}

```

Output Class

```

apex

public class ContactOutput {
    @InvocableVariable
    public String greetingMessage;
}

```

Invocable Method Returning Output

```

apex

public class ContactGreeting {
    @InvocableMethod(label='Generate Greetings')
    public static List<ContactOutput> generateGreetings(List<ContactInput> inputs) {
        List<ContactOutput> results = new List<ContactOutput>();
        List<Contact> cons = [SELECT Id, FirstName FROM Contact WHERE Id IN :inputs];
        Map<Id, String> idToName = new Map<Id, String>();
        for (Contact c : cons) {

```

```

        idToName.put(c.Id, c.FirstName);
    }

    for (ContactInput input : inputs) {
        ContactOutput output = new ContactOutput();
        output.greetingMessage = 'Hello ' + idToName.get(input.contactId) + '!';
        results.add(output);
    }

    return results;
}

```

✓ This lets your Flow **receive greetings for each Contact**.

Best Practices

- ✓ Use **custom types** (Input/Output classes) for clarity.
- ✓ Keep logic small and focused.
- ✓ Annotate fields with **@InvocableVariable**.
- ✓ Always bulkify—your method gets called with **lists**, not single records.
- ✓ Use labels and descriptions for clarity in Flow Builder.

✓ Quick Recap

Invocable Methods let you:

- Expose Apex logic to declarative tools.
- Accept and return structured data.
- Empower admins without them writing Apex code.

✓ If you'd like, I can help you:

- Write a test class for an invocable method.
- Create more advanced examples.
- Combine with asynchronous Apex or triggers.

Just let me know!



Explain test classes in detail

Absolutely—let's **deep dive into Apex test classes** in Salesforce:

🌟 What is a Test Class?

A **test class** is a special Apex class you write to:

- ✓ Verify your code (triggers, classes, batches) works correctly.
- ✓ Generate **code coverage** (minimum 75% coverage is required to deploy to Production).
- ✓ Make sure future changes don't break existing logic (regression testing).

Test classes NEVER run in production context (i.e., they don't commit data).

Why Do We Write Tests?

- 1 **Verify functionality:** Make sure your code does what you expect.
- 2 **Code coverage:** Salesforce requires 75% coverage across Apex code.
- 3 **Enforce best practices:** Good tests document how your code should be used.
- 4 **Data isolation:** Tests run in a special environment and don't affect real data.

✓ Important Features of Apex Testing

Feature	Meaning
@isTest	Marks a class or method as a test.
Test.startTest() / stopTest()	Used to reset governor limits and test asynchronous code.
SeeAllData=false	Test class cannot access org data (recommended).
SeeAllData=true	Test class CAN access org data (not recommended except in very specific scenarios).
Test.setMock()	Use mocks to test HTTP callouts or web services.
System.assert()	Validates expected results.

✨ Syntax of a Simple Test Class

```
apex

@isTest
private class AccountHelperTest {
    @isTest static void testUpdateAccounts() {
        // Prepare test data
        Account acc = new Account(Name='Test Account');
        insert acc;

        // Call your method
        AccountHelper.updateAccountDescription(new List<Id>{acc.Id});

        // Verify
        Account updatedAcc = [SELECT Description FROM Account WHERE Id = :acc.Id];
        System.assertEquals('Updated by Helper', updatedAcc.Description);
    }
}
```

✓ Notes:

- **@isTest:** Marks this as a test class.
- **private** scope is recommended.

- **static** test methods.
- No need for constructors—everything is static.

What is SeeAllData?

- **SeeAllData=false (default):**
 - Test class **cannot see org data**.
 - You must create your own test records.
- **SeeAllData=true:**
 - Test class can see org records.
 - **NOT recommended** because:
 - Makes tests unreliable (data changes).
 - Breaks isolation.

Tip: Always **create your own test data** inside test methods.

✨ Using `Test.startTest()` and `Test.stopTest()`

These are used to:

- ✓ Reset governor limits inside your test.
- ✓ Force execution of asynchronous code (e.g., Future, Queueable, Batch).

Example:

```
apex

@isTest
private class FutureMethodTest {
    @isTest static void testFutureMethod() {
        // Setup data
        Account acc = new Account(Name='Future Account');
        insert acc;

        // Start test context
        Test.startTest();

        // Call your @future method
        FutureHelper.updateDescriptionFuture(new List<Id>{acc.Id});

        // Stop test context (this makes future method run synchronously)
        Test.stopTest();

        // Verify
        Account updated = [SELECT Description FROM Account WHERE Id = :acc.Id];
        System.assertEquals('Updated by Future', updated.Description);
    }
}
```

- ✓ Always wrap asynchronous tests with **startTest()** and **stopTest()**!

✨ Test Class Example for Trigger Handler

Suppose you have this trigger handler:

```
apex

public class AccountTriggerHandler {
    public static void beforeInsert(List<Account> newList) {
        for (Account acc : newList) {
            acc.Description = 'Default Description';
        }
    }
}
```

And this trigger:

```
apex

trigger AccountTrigger on Account (before insert) {
    if (Trigger.isBefore && Trigger.isInsert) {
        AccountTriggerHandler.beforeInsert(Trigger.new);
    }
}
```

✓ Test Class:

```
apex

@Test
private class AccountTriggerHandlerTest {
    @Test static void testBeforeInsert() {
        // Create test record
        Account acc = new Account(Name='Trigger Test');

        // Insert (will fire trigger)
        insert acc;

        // Verify
        Account inserted = [SELECT Description FROM Account WHERE Id = :acc.Id];
        System.assertEquals('Default Description', inserted.Description);
    }
}
```

✨ Example Testing Batch Apex

Batch Apex tests must:

- ✓ Create enough data.
- ✓ Enqueue the batch.
- ✓ Use `Test.startTest()` and `Test.stopTest()`.

```
apex

@Test
private class AccountBatchTest {
    @Test static void testBatch() {
        List<Account> accs = new List<Account>();
        for (Integer i=0; i<200; i++) {
            accs.add(new Account(Name='Batch ' + i));
        }
        insert accs;

        Test.startTest();
        AccountBatchProcessor batch = new AccountBatchProcessor();
    }
}
```

```

        Id batchId = Database.executeBatch(batch);
        Test.stopTest();

        // Verify data updated
        Integer updatedCount = [SELECT count() FROM Account WHERE Description = 'Processed by
Batch'];
        System.assertEquals(200, updatedCount);
    }
}

```

✨ Example Testing Queueable Apex

```

apex

@isTest
private class QueueableTest {
    @isTest static void testQueueable() {
        // Setup
        Account acc = new Account(Name='Queueable');
        insert acc;

        // Run Queueable
        Test.startTest();
        System.enqueueJob(new QueueableAccountUpdater(new List<Id>{acc.Id}));
        Test.stopTest();

        // Verify
        Account updated = [SELECT Description FROM Account WHERE Id = :acc.Id];
        System.assertEquals('Updated by Queueable', updated.Description);
    }
}

```

✨ System.assert() and System.assertEquals()

These statements **validate your results**:

Method	Usage
System.assert(condition)	True/false check
System.assertEquals(expected, actual)	Compares expected to actual
System.assertNotEquals()	Verifies values are different

✓ Always include assertions—they **prove your logic works**.

✨ Tips for Writing Good Tests

- ✓ Create all needed data inside the test.
- ✓ Never rely on org data (unless absolutely necessary).
- ✓ Cover positive, negative, and edge cases.
- ✓ Use **Test.startTest()** / **stopTest()** for async.
- ✓ Use **System.assert()** to validate behavior.
- ✓ Keep methods small and descriptive.
- ✓ Strive for **100% coverage**, not just 75%.

🚩 Recap: Checklist for Every Test Class

- ✓ Annotate with @isTest
- ✓ No constructors—all methods static
- ✓ Prepare all data needed
- ✓ Call your logic
- ✓ Use assertions
- ✓ Cover all branches (if, else, error scenarios)

✓ If you'd like, I can help you:

- Write test classes for your triggers, batches, or queueables.
- Show advanced mocking (e.g., HTTP callouts).
- Review and improve existing tests.

Just share your code—I'm here to help!



Trigger order of execution

Great question—**Trigger Order of Execution** is a core Salesforce concept that explains **what happens and in which order when you insert/update/delete a record**.

Below is a **detailed, step-by-step explanation** you can easily reference or include in your notes.

★ Trigger Order of Execution

When you **save a record (insert/update/delete/upsert)**, Salesforce performs a **specific sequence of events** behind the scenes.

This ensures:

- Validation rules,
- Workflows,
- Processes,
- Triggers,
- Roll-up summaries,
- Assignment rules,

...all run in a consistent and predictable order.

🔧 Step-by-Step Order

Let's look at the **full sequence**:

1 Original Record Loaded

- Loads the **existing record from the database** (or initializes a new record for inserts).

2 System Validation Rules

- Validates:
 - Required fields.
 - Field formats (e.g., picklist values).
 - Data types.
 - **NOT custom validation rules yet.**

3 Before Triggers

- Executes any **before insert**, **before update**, or **before delete** triggers.
- This is where you can **modify field values before saving**.

✓ Example:

```
apex

trigger AccountTrigger on Account (before insert) {
    for(Account a : Trigger.new){
        a.Description = 'Set in before trigger';
    }
}
```

4 Custom Validation Rules

- Runs **your custom validation rules** configured in Setup.

✓ Example:

```
javascript

Error: Annual Revenue must be greater than zero.
```

5 Duplicate Rules

- Checks duplicate management rules (if enabled).

6 Record Saved to Database (but not committed yet)

- Performs the **DML operation** in memory.

- This is when the **record gets an Id** if inserting.

7 After Triggers

- Executes **after insert**, **after update**, or **after delete** triggers.
- Here, you **cannot modify the record** directly.

✓ Example:

```
apex

trigger ContactAfterInsert on Contact (after insert) {
    for(Contact c : Trigger.new){
        Task t = new Task(
            Subject = 'Follow up',
            WhatId = c.Id,
            ActivityDate = Date.today().addDays(3)
        );
        insert t;
    }
}
```

8 Assignment Rules

- Runs (e.g., assign Case or Lead owners).

9 Auto-Response Rules

- Sends auto-response emails (e.g., web-to-case).

10 Workflow Rules

- Evaluates workflow rules.
- If criteria met:
 - Field Updates.
 - Email Alerts.
 - Tasks.
 - Outbound Messages.

✓ Important:

If a **workflow field update** happens:

- Salesforce **updates the record again**.
- **BEFORE UPDATE** triggers fire again.
- **AFTER UPDATE** triggers fire again.
- Validation rules **do NOT run again**.
- Workflow rules are re-evaluated (but only once more).

1 1 Processes and Flows

- Runs **Process Builder** flows.
- May again update records, causing another round of **before update/after update** triggers.

1 2 Escalation Rules

- For Cases—evaluates escalation.

1 3 Roll-Up Summary Fields

- Recalculates roll-up summaries (if Master-Detail).

1 4 Criteria-Based Sharing Rules

- Recalculate sharing rules.

1 5 Commit to Database

- All changes are **committed to the database**.

1 6 Post-Commit Logic

- Sends:
 - Email notifications.
 - Outbound messages.
 - Executes asynchronous events (like @future methods).

✓ Tip to Remember:

Before triggers: Modify field values.

After triggers: Access Ids and related objects, but **can't change Trigger.new directly**.

Example: Trigger Chain for INSERT

- 1 **before insert** triggers.
- 2 Validation rules.
- 3 Record saved (with Id assigned).
- 4 **after insert** triggers.
- 5 Workflow field updates (if any) -> re-run **before update** and **after update** triggers.
- 6 Process Builder (may also update).
- 7 Commit.
- 8 Send emails, fire async jobs.

Why This Matters

- Prevent **recursive triggers** by controlling updates.
- Be aware of **workflow field updates** causing **trigger re-entry**.
- Understand that **validation runs BEFORE triggers**, but not after workflows.
- Know **when IDs are available** (after `before insert`).

Pro Tip: Prevent Recursive Updates

Use a static variable to avoid infinite loops:

```
apex

public class MyTriggerHandler {
    public static Boolean isAlreadyRun = false;

    public static void afterUpdate(List<Account> accounts) {
        if(isAlreadyRun) return;
        isAlreadyRun = true;

        // Logic that might update the same object
    }
}
```

✅ That's the **Trigger Order of Execution**—one of the most important topics in Salesforce Apex development.

If you'd like, I can help you:

- Create diagrams for this flow.
- Explain how it impacts real-world triggers.
- Help you debug tricky recursion issues.

Just let me know!